Keywords: reinforcement learning • reward machines • vision-language models • non-Markovian tasks

RM-VLM: From Natural-Language Task Descriptions to Vision-Grounded Reward Machines

An LLM Builds the Finite Automaton; a VLM Grounds Its Propositions; the Agent Earns Credit for History-Dependent Task Progress

By Roko Parac, Christos Ziakas, Leo Ardon

arXiv:2611.08308 • cs.AI • submitted 6 Nov 2026

Most VLM-based reward methods assume that reward depends only on the current observation. RM-VLM handles tasks that require history-dependent credit by having an LLM construct a reward machine from a text description and a VLM ground each proposition in image observations. Evaluated on grid navigation, robotic manipulation, and 3D object collection, RM-VLM consistently outperforms existing VLM-reward methods on task success.

AT A GLANCE

Problem: VLM reward methods assume Markovian tasks; real tasks often require ordered sub-goals whose credit depends on history.

Why it matters: Without history-aware reward, RL agents fail to distinguish correct-order completions from incorrect-order ones and cannot learn ordered, multi-step tasks reliably.

Method: LLM decomposes task description into propositions and builds a reward machine (finite automaton); VLM grounds propositions in image observations; VLM value estimator provides dense shaping.

Result: Consistently higher task success than all baselines across three task domains.

Evidence: Grid-world navigation, robotic manipulation sequence, 3D object collection; baselines: VLM-RM, CLIP-reward, TAMER+VLM.

The Big Picture

Reinforcement learning agents historically required handcrafted reward functions that encode both the goal and the intermediate milestones. VLM-based reward models automate this by scoring image observations against a natural-language goal description: an agent that moves closer to the goal receives a higher reward from the VLM, without any manual reward engineering.

This works well for Markovian goals where the reward depends only on the current state. It breaks for *ordered* tasks: "pick up the red block, then place it in the green zone, then open the door." A VLM reward that evaluates only the current image cannot distinguish whether the red block was placed *before* or *after* the green zone interaction. An agent that learns to shortcut to the door without completing earlier sub-goals will receive high reward on the final state.

Reward machines (Icarte et al., 2018) are the standard formalism for non-Markovian task structure: a finite automaton tracks which sub-goals have been achieved and transitions on logical propositions evaluated in the environment. RM-VLM combines the LLM's ability to decompose natural-language task descriptions with the VLM's ability to ground propositions in images.

Why Existing Approaches Fall Short

VLM scalar reward (VLM-RM) scores the current image against the full task description. It cannot distinguish sub-goal ordering and provides identical reward to agents that complete sub-goals in any order.

CLIP-reward embeds both image and text goal in CLIP space and uses cosine similarity as reward. It has the same Markovian limitation and additionally fails to track incremental progress toward structured sub-goals.

TAMER with VLM feedback uses intermittent human-style feedback modeled by a VLM. The feedback is episodic and noisy; it does not provide dense, automaton-guided credit for individual transitions.

Handcrafted reward machines (prior RM work) require a domain expert to specify the propositions and automaton structure manually — the bottleneck RM-VLM eliminates.

Core Mechanism

Step 1: Reward machine construction. The task description $\mathcal{T}$ is passed to an LLM with a structured prompt requesting:

- A set of Boolean propositions $\mathcal{P} = \{p_1, \ldots, p_n\}$ over environment entities.
- A finite automaton $M = (Q, q_0, q_f, \delta, \rho)$ where $\delta : Q \times 2^{\mathcal{P}} \to Q$ is the transition function and $\rho : Q \times Q \to \mathbb{R}$ assigns reward to transitions.

The LLM outputs M as a JSON object, which is validated and compiled.

Step 2: VLM grounding. At each step, the VLM receives the current image observation o_t and each proposition p_i, returning $P(p_i{=}\text{true} \mid o_t)$. The resulting Boolean vector advances M's state through δ, triggering the reward $\rho(q_t, q_{t+1})$ on any transition.

Step 3: Dense reward shaping. To alleviate sparse

rewards in long-horizon tasks, a VLM-based value estimator V_{VLM} scores the image against the proposition set of each pending transition, producing a shaped bonus:

$$r_t^+ = \lambda \sum_{p_i \in \text{pending}(q_t)} P(p_i{=}\text{true} \mid o_t),$$

where λ is a shaping coefficient and $\text{pending}(q_t)$ is the set of propositions required to advance from q_t.

Technical Formulation

The reward machine $M = (Q, q_0, q_f, \delta, \rho)$ defines a non-Markovian MDP $\mathcal{M} = (S, A, T, R_M)$ where the total reward at step t is:

$$R_M(s_t, a_t) = \rho(q_t, q_{t+1}) + r_t^+,$$

with $q_{t+1} = \delta(q_t, L(o_t))$ and $L(o_t) \in 2^{\mathcal{P}}$ the labelling function implemented by the VLM. The augmented state (s_t, q_t) is Markovian in $\mathcal{M}$, so standard policy-gradient methods (PPO, SAC) apply.

The labelling function uses a threshold $\tau = 0.5$:

$$L(o_t) = \{p_i : P(p_i{=}\text{true} \mid o_t) \geq \tau\}.$$

Learning or Inference Procedure

1. Submit task description $\mathcal{T}$ to LLM to obtain M as JSON.
2. Validate M (check that q_f is reachable, no duplicate states, all propositions well-formed).
3. Initialize policy π_θ with PPO; use (s_t, q_t) as the augmented state.
4. At each environment step, query VLM for $L(o_t)$, advance M, and compute $R_M(s_t, a_t)$.
5. Train until convergence (5M environment steps per domain).

No fine-tuning of the LLM or VLM is performed; both are used frozen.

What the Guarantee Says

No formal guarantee is claimed. The empirical claim is consistent task-success superiority over all baselines in all three domains tested. No sample complexity bound is given; the 5M-step budget is task-specific and may not be sufficient for harder domains.

Experimental Findings

- **Grid-world navigation:** RM-VLM task success 87% vs. VLM-RM 61%, CLIP-reward 54%, TAMER+VLM 70%.
- **Robotic manipulation sequence:** RM-VLM 74% vs. VLM-RM 38%, CLIP 29%, TAMER+VLM 51%; largest gap because out-of-order completions are highest in this domain.
- **3D object collection:** RM-VLM 81% vs. VLM-RM 58%.
- **Dense shaping:** Removing the shaped bonus r_t^+ reduces robotic manipulation success from 74% to 52%, confirming its role in long-horizon credit propagation.

Ablations and Interpretation

LLM construction errors: On 15% of randomly sampled task descriptions, the LLM produces a machine with at least one missing transition or unreachable state. These cases are caught by validation and trigger a retry; the retry resolves 90% of errors.

VLM grounding accuracy: Replacing the VLM labelling function with ground-truth oracle labels improves manipulation success from 74% to 88%, establishing an upper bound from grounding accuracy limitations.

Threshold τ: Grid search over $\tau \in \{0.3, 0.5, 0.7\}$ shows $\tau = 0.5$ optimal across all three domains; $\tau = 0.3$ causes false-positive transitions; $\tau = 0.7$ misses valid ones.

Shaping coefficient λ: $\lambda = 0.1$ is optimal; larger values cause the agent to maximize the shaped bonus rather than completing transitions, a known reward-shaping pathology.

Reference

Roko Parac, Christos Ziakas, Leo Ardon. **RM-VLM: From Natural-Language Task Descriptions to Vision-Grounded Reward Machines.** arXiv:2611.08308. https://arxiv.org/abs/2611.08308